Writing ledger.py yourself

A study guide to `core/ledger.py`: the thinking, the Python, and the pictures behind it

What you'll be able to do after this

- Explain what every line of `ledger.py` does, and why it's there.
- Rebuild the file from an empty editor in seven steps, testing each one.
- Use the same thinking for your next helper: start from the contract, make it work plainly, then add one concern at a time.
- Recognise and use the Python tools inside it: `importlib`, `inspect.signature`, `dict.setdefault`, `**kwargs`, Polars lazy filters, `pl.collect_all`, `threading.Lock`, `warnings.catch_warnings`, and more.

How to use it

- Read Parts 1 and 2 once: the problem, the mindmap and the flow.
- Work through Part 3 with your editor open. Type the code yourself; don't paste it.
- Do the checkpoints in Part 6 from memory, and only look back when stuck.
- Keep Part 4 (the toolbox) and the annotated listing at the end as a reference.

Every code snippet in this guide was run before it was written down: the step versions against test ledger files shaped like yours, and the checkpoint tests against the real project.

Part 1. Think before you write code

The most important part of writing a helper like this happens before the first line of code. You decide **what it must do for its callers**, and you list the facts about the world it has to cope with. Each fact becomes one piece of the code.

The job, in one sentence

> **ledger.py**
> Give any report the ledger rows it needs (one date range, some clients, some ledger types) as a pandas table, plus any warnings worth showing the user, without loading the whole ledger.

The contract: write this first

A contract is the function's name, its inputs and its output. Writing it first, even with no body, settles what the rest of the code must achieve:

```
def get_ledger_slice(start, end, clients=None, types=None):
    """Rows with start <= CreatedAt < end, optionally only some clients and ledger types.

    Returns (rows as a pandas DataFrame, notes): notes are warnings worth showing on the page.
    """
    ...
```

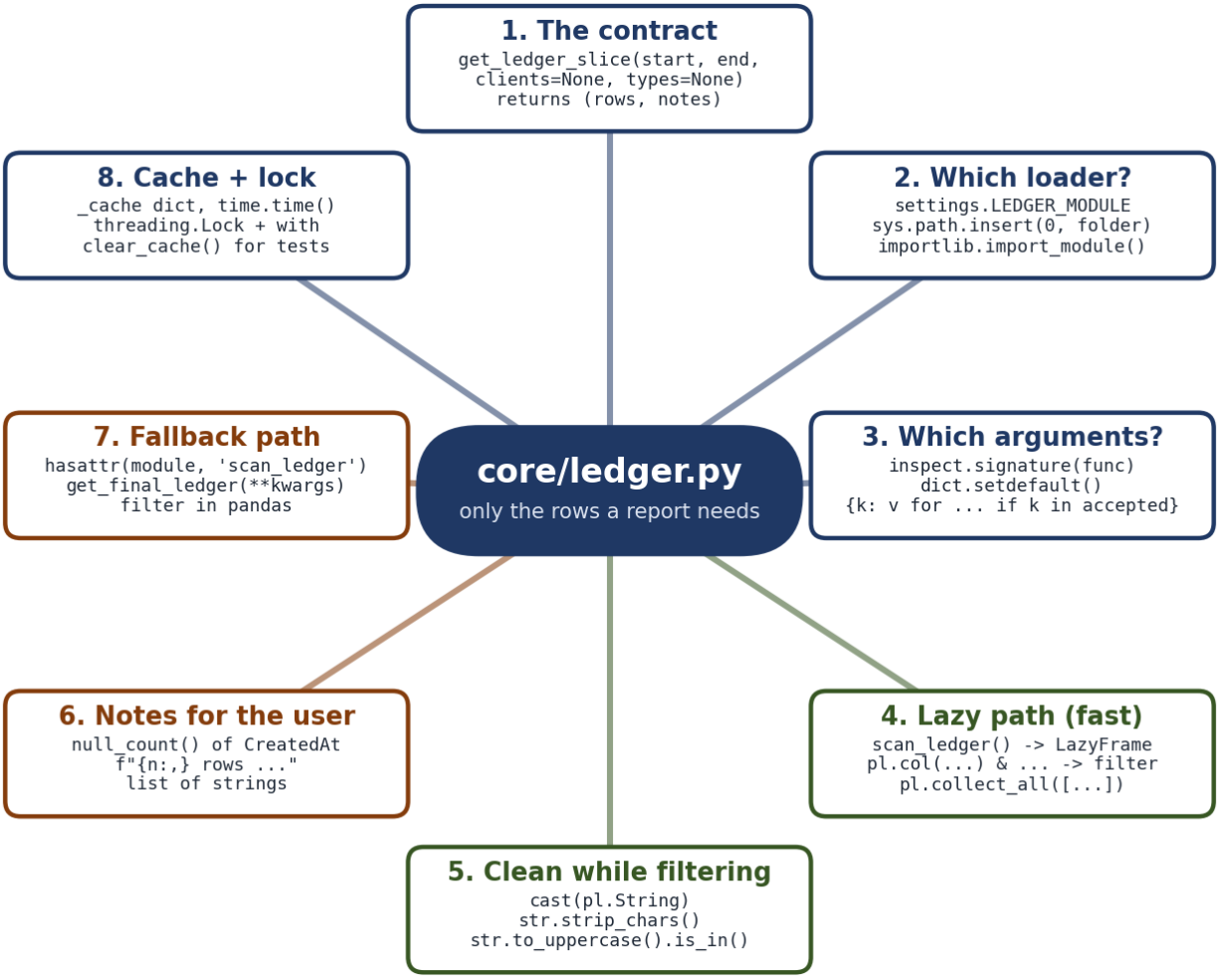
- `start`, `end`: a half-open range, the same rule as in your notebooks: `>= start` and `< end`.
- `clients=None`, `types=None`: **optional** filters. `None` means "don't filter on this".
- **Returns a tuple**, `(rows, notes)`. The caller unpacks it: `led, notes = get_ledger_slice(...)`.

Five facts that shape the design

Fact about the world	So the code must...	Piece of code
The ledger is millions of rows in CSVs	never load all of it; filter while reading	Polars lazy scan + filter, then collect
The loader differs by setting: the demo, your real loader, fakes in tests	not hard-code which loader it imports	_module(): settings + importlib
Loaders accept different arguments	pass only what each one accepts	_loader_kwargs(): inspect.signature
Real data is messy: 'bill ', 'C100 ', IDs read as numbers	clean while filtering	cast, strip_chars, to_uppercase
Users never see the server log	return warnings as text the page can show	notes list, null_count, collect_all
An older loader may only have get_final_ledger()	still work, without reloading every time	hasattr fallback, _cache, Lock

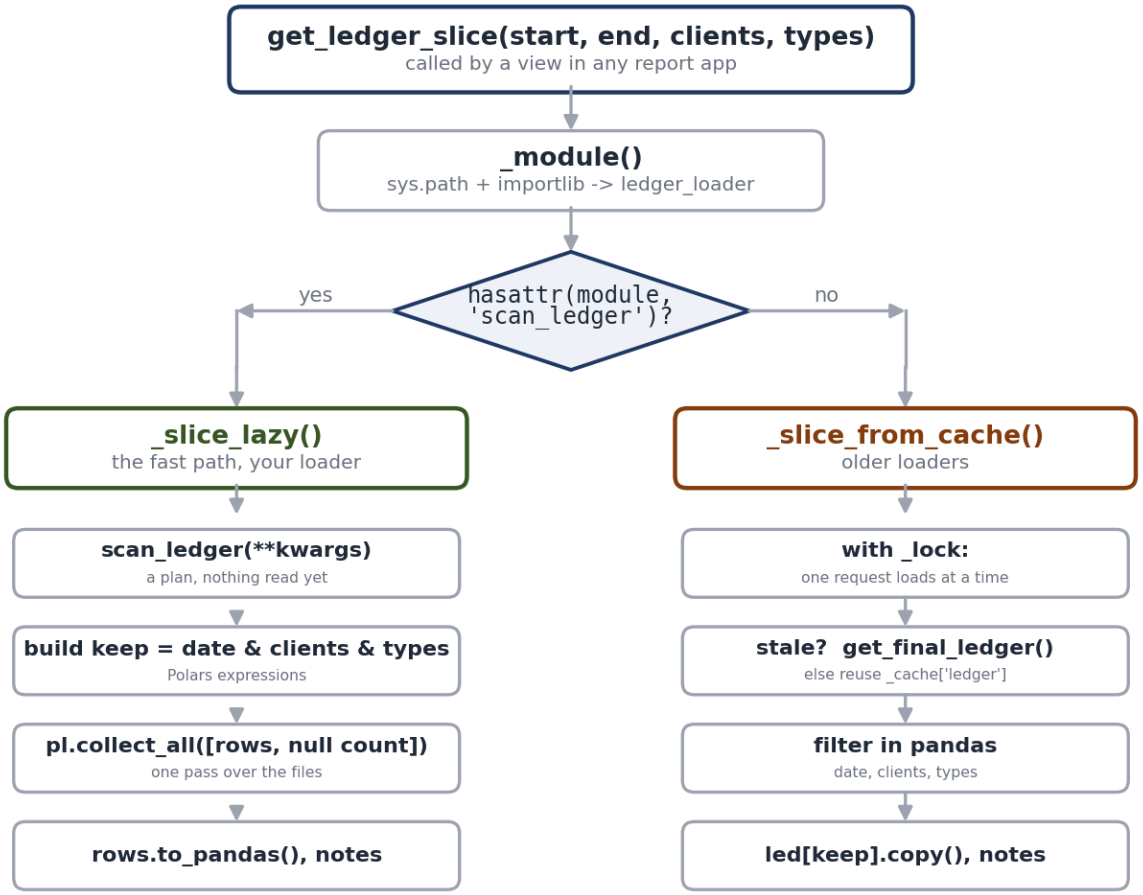
The whole file as a mindmap

Each branch is one question the code answers. The colours match the rest of this guide: **navy** for set-up, **green** for the fast path, **brown** for the fallback and the user-facing notes.



Part 2. The big picture

One public function, four private helpers. A view only ever calls `get_ledger_slice`; everything else is an internal detail, marked by the leading underscore.



Both paths return the same thing: (pandas DataFrame of rows, list of note strings)

Who does what

Function	Job	Called by	Returns
`get_ledger_slice`	The front door: picks the fast or the fallback path	views in every app	(rows, notes)
`_module`	Imports the loader module named in settings	get_ledger_slice	a module object
`_loader_kwargs`	Keeps only the arguments a loader function accepts	both slice paths	a dict
`_slice_lazy`	Filters the lazy plan, then collects once	get_ledger_slice	(rows, notes)
`_slice_from_cache`	Loads the full ledger once, caches it, filters in pandas	get_ledger_slice	(rows, notes)
`clear_cache`	Forgets the cached ledger	tests	None

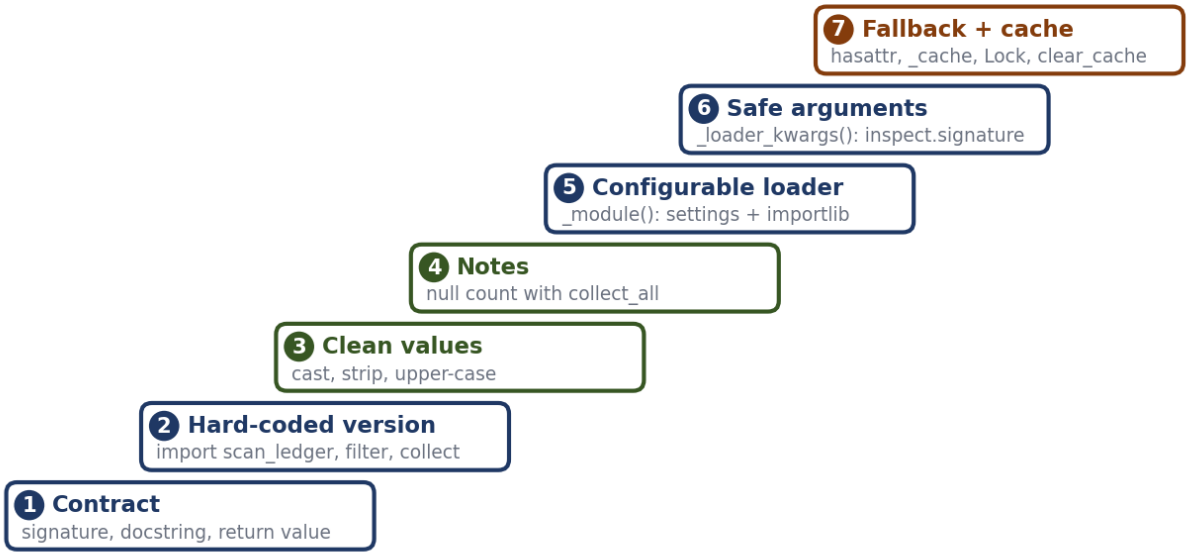
Reading order

Python files are read top to bottom when imported, but you understand them best in the order a call flows: `get_ledger_slice` → `_module` → `_slice_lazy` (→ `_loader_kwargs`). Read the fallback last; it's only for older loaders.

Part 3. Build it in seven steps

Nobody writes this file top to bottom in one go. It was built in layers: first a plain version that works, then one concern at a time. Each step below has the thinking, the code, the Python ideas it uses, and a check you can run.

> *Each step works and is tested before the next one starts.*



Step 1. The contract

What I was thinking
"Views shouldn't know anything about CSVs, Polars or loaders. They should ask one question, 'give me these rows', and get a pandas table back."
"Optional filters default to None, so the simplest call is just a date range."
"Warnings need to reach the page, so return them alongside the rows rather than printing them."

```
def get_ledger_slice(start, end, clients=None, types=None):
    """Rows with start <= CreatedAt < end, optionally only some clients and ledger types.

    Returns (rows as a pandas DataFrame, notes): notes are warnings worth showing on the page.
    """
    rows = pd.DataFrame()           # placeholder until step 2
    notes = []
    return rows, notes
```

Python idea	What it means here
Default None	`clients=None`: the caller can leave it out. Inside, `if clients is not None:` decides whether to filter.
Docstring	The text in triple quotes. It's what Shift+Tab shows in a notebook, and it states the contract.
Returning a tuple	`return rows, notes` returns two values; the caller writes `led, notes = ...`.

Check yourself
Call it with two dates and print `type(result)` and `len(result)`. You should see a tuple of 2.

Step 2. A hard-coded version that works

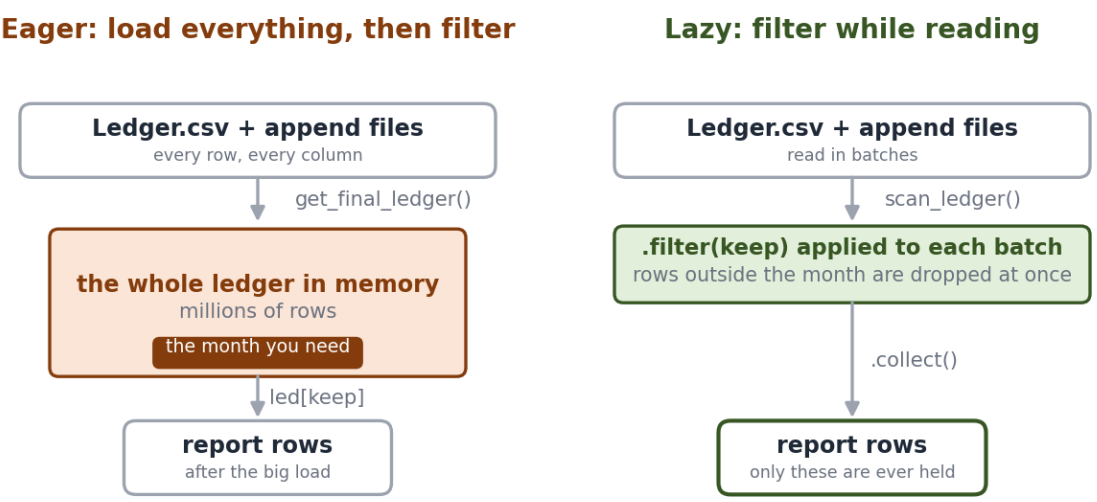
What I was thinking
"Make it work first, plainly. Import the loader directly, no settings, no fallback."
"`scan_ledger()` gives me a *plan*, not data. If I filter the plan before `.collect()`, Polars only keeps the rows that pass while it reads."
"Start with the date condition, then add the optional ones, one at a time."

v1: tested on files shaped like yours

```
import pandas as pd
import polars as pl
from ledger_loader import scan_ledger

COLUMNS = ("ClientID", "CreatedAt", "LedgerType", "Debit", "Credit")


def get_ledger_slice(start, end, clients=None, types=None):
    lf = scan_ledger(columns=COLUMNS)                                    # a plan: nothing read yet
    keep = (pl.col("CreatedAt") >= start) & (pl.col("CreatedAt") < end)
    if clients is not None:
        keep = keep & pl.col("ClientID").is_in(list(clients))
    if types is not None:
        keep = keep & pl.col("LedgerType").is_in(list(types))
    return lf.filter(keep).collect().to_pandas(), []
```



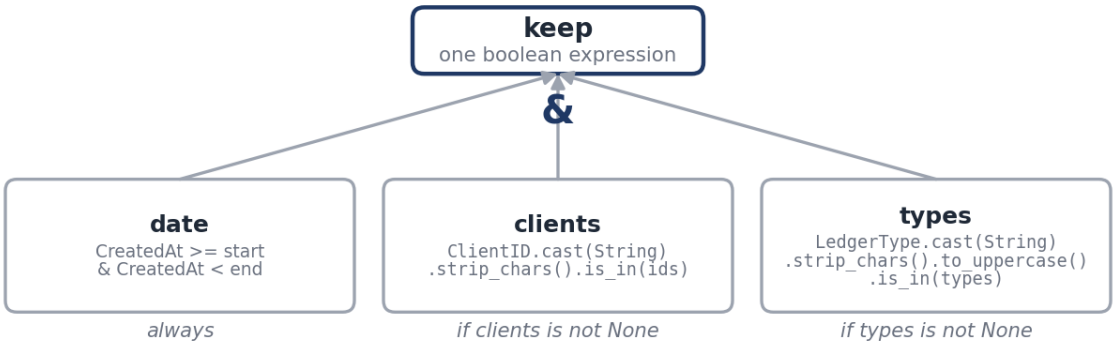
Tool	What it does	Why here
`pl.col('CreatedAt')`	Refers to a column inside a Polars plan	Conditions are built before any data exists
`(a >= x) & (a < y)`	Combines two conditions with AND	The half-open date range; brackets are required
`keep = keep & ...`	Adds one more condition to the expression	Optional filters are added only when asked for
`.is_in(list)`	True where the value is in the list	Clients and ledger types
`lf.filter(keep)`	Adds the filter to the plan	Still nothing read
`.collect()`	Runs the plan and returns a Polars DataFrame	The only place the files are read
`.to_pandas()`	Converts to pandas	The report code is pandas

Check yourself
Compare with the slow way: `get_final_ledger(as_pandas=True)` filtered in pandas. The same rows should come back. (On test files shaped like yours, they did.)
Passing `clients` as a list, a set, or a pandas Series all work, because of `list(clients)`.

Step 3. Clean values while filtering

What I was thinking
"In real data, a ledger type can be `'bill '` and an ID can be `'C100 '`, or read as a number. An exact match would quietly drop those rows."
"Clean the column inside the filter, not in a separate pass, so it still happens while reading."
"Clean the caller's list too: strip spaces, skip blanks, make each ID a string."

```
if clients is not None:
    ids = sorted({str(c).strip() for c in clients if pd.notna(c)})
    keep = keep & pl.col("ClientID").cast(pl.String).str.strip_chars().is_in(ids)
if types is not None:
    keep = keep & (pl.col("LedgerType").cast(pl.String)
                   .str.strip_chars().str.to_uppercase().is_in(list(types)))
```




```
keep = date;   keep = keep & clients;   keep = keep & types   ->   lf.filter(keep)
```

Tool	What it does	Why here
`{str(c).strip() for c in clients ...}`	A set comprehension: each item, cleaned, no repeats	Uploaded lists can repeat IDs
`if pd.notna(c)`	Skips blanks (NaN, None)	str(NaN) would become the text 'nan'
`sorted(...)`	A list in a fixed order	Same input, same plan, easier to debug
`.cast(pl.String)`	Turns the column into text	IDs read as numbers would never match text
`.str.strip_chars()`	Removes spaces at both ends	'C100 ' becomes 'C100'
`.str.to_uppercase()`	Upper-cases text	'bill' matches 'BILL'
`Timestamp.to_pydatetime()`	A plain Python datetime	Every Polars version accepts it in comparisons

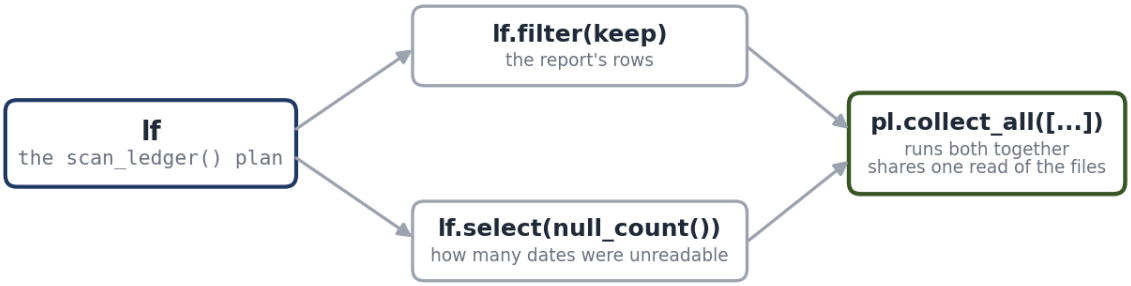
The final file also wraps the dates: `pd.Timestamp(start).to_pydatetime()`. Current Polars accepts a pandas Timestamp directly, but converting makes the function accept a string, a Timestamp or a datetime, on any Polars version.

Check yourself
Build a two-row Polars frame with `' bill'` and `'C1 '`, pass it through a fake `scan_ledger`, and ask for `types=['BILL']`, `clients=['C1']`. One row should come back (checkpoint B in Part 6).

Step 4. Notes for the user

What I was thinking
"`get_final_ledger` warns about unreadable dates, but on a web server nobody reads warnings. The report page should show them."
"Rows with no CreatedAt fail every date comparison, so they silently vanish from every report. Count them so the user knows."
"Counting means scanning all the files again, unless both questions run together: `pl.collect_all`."

```
rows, bad = pl.collect_all([lf.filter(keep),
                            lf.select(pl.col("CreatedAt").null_count().alias("n"))])
notes = []
n_bad = int(bad["n"][0])
if n_bad:
    notes.append(f"{n_bad:,} ledger rows have an unreadable CreatedAt and were left out of every report.")
return rows.to_pandas(), notes
```



`rows, bad = pl.collect_all([lf.filter(keep), lf.select(pl.col('CreatedAt').null_count())])`

Tool	What it does	Why here
`.null_count()`	How many values are missing	Unreadable dates are stored as null by the loader
`.alias('n')`	Names the result column	So it can be read as bad['n']
`lf.select(...)`	A plan that returns only that calculation	One number, not the rows
`pl.collect_all([a, b])`	Runs several plans together, sharing work	One read of the files, two answers
`bad['n'][0]`	The first (only) value of column n	A one-row result
`int(...)`	A plain Python int	Clean for if-checks and f-strings
`f"{n_bad:,}"`	Formats with thousands separators	1234 becomes 1,234

Check yourself
Make a fake loader with one row whose CreatedAt is None. The result should have one fewer row, and one note mentioning "1 ledger rows".

Step 5. Make the loader configurable

What I was thinking
"`from ledger_loader import scan_ledger` at the top of the file fixes one loader forever. The demo, your real loader and test fakes all need to plug in."
"Settings already say where things are. Read the module's *name* from settings and import it by name."
"If the module lives in another folder, add that folder to Python's search path first, the same trick as in your notebooks."

```
import importlib
import sys
from django.conf import settings


def _module():
    """Import the loader module named in settings, from LEDGER_MODULE_PATH if given."""
    folder = getattr(settings, "LEDGER_MODULE_PATH", "")
    if folder and folder not in sys.path:
        sys.path.insert(0, folder)
    return importlib.import_module(settings.LEDGER_MODULE)
```

Tool	What it does	Why here
`getattr(settings, 'X', '')`	Reads a setting, with a default if it's missing	An optional setting never crashes
`folder not in sys.path`	Checks the list before adding	Without it, every call would add the folder again
`sys.path.insert(0, folder)`	Searches this folder first	Your ledger_loader.py wins over any other copy
`importlib.import_module(name)`	Imports a module whose name is in a variable	'core.demo_ledger' or 'ledger_loader'
`_module (leading _)`	"Private: used inside this file only"	Views shouldn't call it

`importlib.import_module` also uses the `sys.modules` cache: the second call returns the already-imported module instantly, so calling it on every request is cheap.

Check yourself
With `LEDGER_MODULE = "core.demo_ledger"`, `_module().__name__` is `"core.demo_ledger"`. Change the setting and it changes, with no code edit.

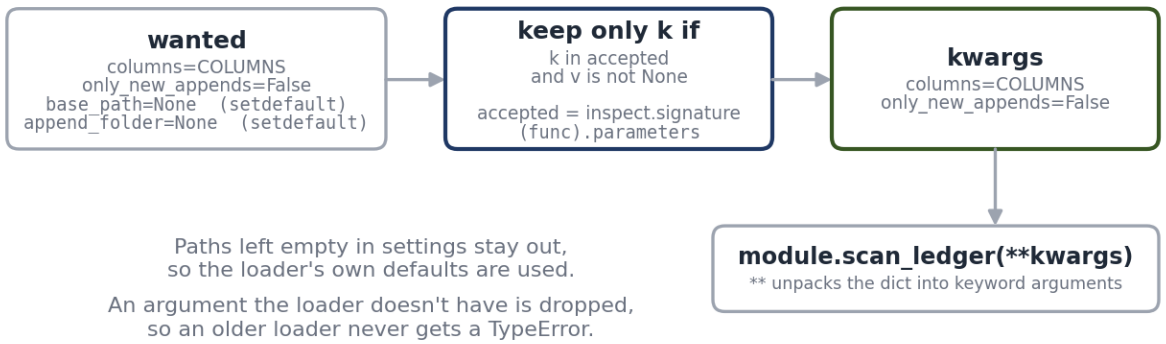
Step 6. Pass only the arguments a loader accepts

What I was thinking
"Your loader takes `base_path` and `append_folder`; the demo doesn't. Passing an argument a function doesn't have raises `TypeError: got an unexpected keyword argument`."
"So: build everything I'd like to pass, then ask the function which parameters it has, and keep only those."
"Paths left empty in settings should not be passed at all, so the loader's own defaults apply."

```
import inspect


def _loader_kwargs(func, **wanted):
    """Pass only the arguments this loader function accepts, and only paths that are set in settings."""
    wanted.setdefault("base_path", getattr(settings, "LEDGER_BASE_PATH", "") or None)
    wanted.setdefault("append_folder", getattr(settings, "LEDGER_APPEND_FOLDER", "") or None)
    accepted = inspect.signature(func).parameters
    return {k: v for k, v in wanted.items() if k in accepted and v is not None}


# used like this:
kwargs = _loader_kwargs(module.scan_ledger, columns=COLUMNS, only_new_appends=False)
lf = module.scan_ledger(**kwargs)
```



Tool	What it does	Why here
`def f(func, **wanted)`	Collects any keyword arguments into a dict	Callers pass whatever they'd like to send
`wanted.setdefault(k, v)`	Adds k only if it isn't there yet	Callers can still override the paths
`"" or None`	Empty text becomes None	None later means "don't pass it"
`inspect.signature(func).parameters`	The function's parameter names	Asks the loader what it accepts
`{k: v for ... if ...}`	A dict comprehension with a filter	Keeps accepted, non-empty arguments
`func(**kwargs)`	Unpacks a dict into keyword arguments	Calls the loader with exactly those

Edge case worth knowing
A loader written as `def get_final_ledger(**kw)` has one parameter, named `kw`, so this filter passes it **nothing**. That was confirmed with a test. If you ever write a loader like that, extend the filter: pass everything when any parameter's kind is `inspect.Parameter.VAR_KEYWORD`.

Check yourself
Checkpoint E in Part 6: two fake loaders with different parameters get different argument dicts.

Step 7. The fallback, the cache and the lock

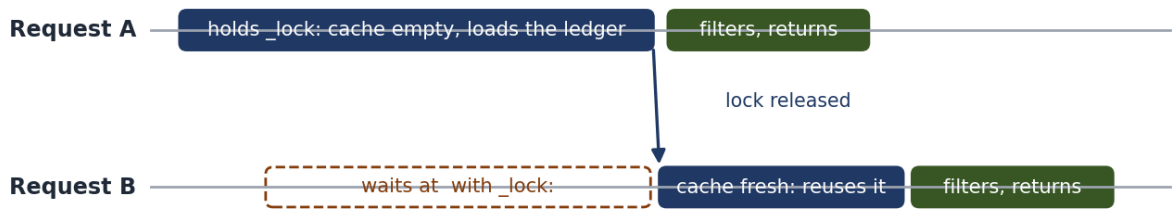
What I was thinking
"A loader without `scan_ledger` can only load everything. Doing that on every request would be painfully slow, so load once and keep it in memory for a while."
"Where does 'keep it' live? In a dict at module level: it survives between requests, because the module stays imported while the server runs."
"Two people clicking at once would both find the cache empty and both load the ledger. A lock makes the second one wait, then reuse what the first loaded."
"Tests need a way to empty the cache: clear_cache()."

```
def get_ledger_slice(start, end, clients=None, types=None):
    module = _module()
    if hasattr(module, "scan_ledger"):                    # new loaders: the fast path
        return _slice_lazy(module, start, end, clients, types)
    return _slice_from_cache(module, start, end, clients, types)


_cache = {"ledger": None, "loaded_at": 0.0, "notes": []}
_lock = threading.Lock()


def _slice_from_cache(module, start, end, clients, types):
    max_age = getattr(settings, "LEDGER_CACHE_MINUTES", 30) * 60
    with _lock:
        if _cache["ledger"] is None or time.time() - _cache["loaded_at"] > max_age:
            kwargs = _loader_kwargs(module.get_final_ledger, columns=COLUMNS, as_pandas=True,
                                    only_new_appends=getattr(settings, "LEDGER_ONLY_NEW_APPENDS", False))
            with warnings.catch_warnings(record=True) as caught:
                warnings.simplefilter("always")
                led = module.get_final_ledger(**kwargs)
            if isinstance(led, pl.DataFrame):
                led = led.to_pandas()
            led["CreatedAt"] = pd.to_datetime(led["CreatedAt"])
            _cache.update(ledger=led, loaded_at=time.time(), notes=[str(w.message) for w in caught])
        led, notes = _cache["ledger"], list(_cache["notes"])

    keep = (led["CreatedAt"] >= start) & (led["CreatedAt"] < end)
    if clients is not None:
        keep &= led["ClientID"].astype(str).str.strip().isin({str(c).strip() for c in clients})
    if types is not None:
        keep &= led["LedgerType"].astype(str).str.strip().str.upper().isin(list(types))
    return led[keep].copy(), notes
```



Without the lock, B would see an empty cache too and load the whole ledger a second time.
After LEDGER_CACHE_MINUTES, time.time() - loaded_at > max_age, and the next request reloads.

Tool	What it does	Why here
`hasattr(module, 'scan_ledger')`	Does the object have this attribute?	Choose the path by what the loader can do ("duck typing")
`_cache = {...} at module level`	State that lives as long as the server process	Remember the loaded ledger between requests
`time.time()`	Seconds since 1970, as a float	Age of the cache: now minus loaded_at
`threading.Lock() + with _lock:`	Only one thread inside the block at a time	Two requests never load at once
`warnings.catch_warnings(record=True)`	Collects warnings into a list instead of printing	Turn loader warnings into notes
`warnings.simplefilter('always')`	Don't hide repeated warnings	Python shows each warning once by default
`isinstance(led, pl.DataFrame)`	Is this object of that type?	Some loaders return Polars, some pandas
`_cache.update(a=..., b=...)`	Sets several keys at once	Ledger, time and notes change together
`list(_cache['notes'])`	A copy of the list	A caller adding to notes can't change the cache
`keep &= ...`	Same as keep = keep & ...	The pandas version of the step 2 pattern
`led[keep].copy()`	An independent table	Edits by the caller can't change the cached ledger

```
def clear_cache():
    """Forget the cached full ledger (only used by loaders without scan_ledger)."""
    with _lock:
        _cache.update(ledger=None, loaded_at=0.0, notes=[])
```

Check yourself
Checkpoint F in Part 6: a fake loader that counts its calls is called only once for two reports.

Part 4. The toolbox: every function in ledger.py

Grouped by where it comes from. The examples are tiny on purpose: try each in a notebook cell.

Python built-ins and syntax

Name	Example	Result
`getattr(obj, name, default)`	`getattr(settings, 'LEDGER_CACHE_MINUTES', 30)`	the setting, or 30
`hasattr(obj, name)`	`hasattr(module, 'scan_ledger')`	True or False
`isinstance(x, type)`	`isinstance(led, pl.DataFrame)`	True or False
`dict.setdefault(k, v)`	`d = {'a': 1}; d.setdefault('a', 9)`	d stays {'a': 1}
`dict.update(...)`	`d.update(a=2, b=3)`	{'a': 2, 'b': 3}
`**kwargs (in def)`	`def f(**kw): return kw; f(x=1)`	{'x': 1}
`**dict (in a call)`	`f(**{'x': 1})`	same as f(x=1)
`set comprehension`	`{s.strip() for s in [' a', 'a ']}`	{'a'}
`dict comprehension`	`{k: v for k, v in d.items() if v}`	only truthy values
`x or None`	`'' or None`	None
`f-string format`	`f'{1234567:,}'`	'1,234,567'
`with (context manager)`	`with _lock: ...`	set up, run, always clean up
`_name`	`_cache, _module`	"private to this file" (a convention)

Standard library

Module.function	Example	Result
`sys.path.insert(0, folder)`	`sys.path.insert(0, r'C:\...\migration')`	folder searched first
`importlib.import_module(name)`	`importlib.import_module('core.demo_ledger')`	the module object
`inspect.signature(f).parameters`	`list(inspect.signature(f).parameters)`	['base_path', 'columns', ...]
`time.time()`	`time.time() - started`	seconds elapsed
`threading.Lock()`	`lock = threading.Lock(); with lock: ...`	one thread at a time
`warnings.catch_warnings(record=True)`	`with ... as caught: warnings.warn('x')`	caught[0].message is 'x'
`warnings.simplefilter('always')`	`inside the with block`	repeated warnings are kept

pandas

Name	Example	Result
`pd.Timestamp(x).to_pydatetime()`	`pd.Timestamp('2026-08-01').to_pydatetime()`	datetime(2026, 8, 1, 0, 0)
`pd.notna(x)`	`pd.notna(float('nan'))`	False
`pd.to_datetime(s)`	`pd.to_datetime(led['CreatedAt'])`	datetime column
`.astype(str).str.strip()`	`s.astype(str).str.strip()`	clean text
`.isin(collection)`	`s.isin({'C1', 'C2'})`	True/False per row
`df[mask].copy()`	`led[keep].copy()`	an independent table

Polars

Name	Example	Result
`pl.col(name)`	`pl.col('CreatedAt') >= start`	a condition (an expression)
`expr & expr`	`(a >= x) & (a < y)`	both must be true
`.cast(pl.String)`	`pl.col('ClientID').cast(pl.String)`	text column
`.str.strip_chars()`	`pl.col('ClientID').str.strip_chars()`	spaces removed
`.str.to_uppercase()`	`pl.col('LedgerType').str.to_uppercase()`	upper-case text
`.is_in(list)`	`pl.col('LedgerType').is_in(['BILL'])`	True/False per row
`lf.filter(expr)`	`lf.filter(keep)`	a narrower plan
`lf.select(expr)`	`lf.select(pl.col('CreatedAt').null_count())`	a one-value plan
`.null_count().alias('n')`	`pl.col('x').null_count().alias('n')`	missing count, named n
`pl.collect_all([lf1, lf2])`	`rows, bad = pl.collect_all([...])`	both results, one pass
`.to_pandas()`	`rows.to_pandas()`	a pandas DataFrame

Django

Name	Example	Result
`from django.conf import settings`	`settings.LEDGER_MODULE`	the value from settings.py
`override_settings (tests)`	`with self.settings(LEDGER_MODULE='fake_loader'):`	a setting changed for one test

Part 5. Mistakes this code is built to avoid

Most lines in ledger.py exist because of a specific way things go wrong. Knowing the mistake is the fastest way to remember the line.

If you wrote...	What would happen	What ledger.py does instead
`get_final_ledger() then filter in pandas`	The whole ledger in memory for every report	Filter the lazy plan, then collect
`from ledger_loader import scan_ledger (fixed)`	Demo and tests can't swap the loader	importlib.import_module(settings.LEDGER_MODULE)
`scan_ledger(base_path=..., ...) always`	TypeError on loaders without that parameter	_loader_kwargs passes only accepted ones
`Passing base_path='' from empty settings`	The loader tries to open a file called ''	'' or None, and None is never passed
`pl.col('LedgerType').is_in(['BILL'])`	'bill ' rows silently missing	cast, strip_chars, to_uppercase first
`[str(c) for c in clients]`	A blank ID becomes the text 'nan'	if pd.notna(c), in a set
`lf.filter(...).collect() + lf.select(...).collect()`	The files read twice	pl.collect_all([...])
`Leaving loader warnings to print`	Nobody sees them on a server	Return them as notes
`sys.path.insert on every call`	The list grows with every request	if folder not in sys.path
`A cache without a lock`	Two requests load the ledger twice	with _lock:
`return _cache['ledger'][keep]`	Caller edits could change the cache	.copy() and list(...)
`No way to reset the cache`	Tests leak into each other	clear_cache()

Habits behind the code

- **Write the contract first.** Name, inputs, output and a docstring, before any logic.
- **Make it work plainly, then generalise.** v1 imported the loader directly; configuration came later.
- **One concern per step, tested before the next.** Cleaning, notes, configuration, safety, fallback.
- **Push work to where the data is read.** Filter the plan, don't filter a loaded table.
- **Fail usefully.** Turn problems the user can act on into notes; let real errors raise.
- **Keep the public surface small.** One function for callers; everything else private.

Part 6. Rebuild it yourself: checkpoints

Start from an empty `core/ledger.py` (rename the real one to `ledger_reference.py` first). After each checkpoint, run its test. All of these tests pass against the finished file.

Checkpoint A: the plain version (Steps 1 and 2)

Write v1 in a notebook against your real loader, for one month and three clients. Then compare:

```
fast = get_ledger_slice(pd.Timestamp("2026-10-01"), pd.Timestamp("2026-11-01"), clients=ids, types=["BILL"])[0]
full = get_final_ledger(columns=COLUMNS, as_pandas=True)
slow = full[(full.CreatedAt >= "2026-10-01") & (full.CreatedAt < "2026-11-01")
            & full.ClientID.isin(ids) & full.LedgerType.eq("BILL")]
order = list(COLUMNS)
pd.testing.assert_frame_equal(fast.sort_values(order).reset_index(drop=True),
                              slow.sort_values(order).reset_index(drop=True), check_dtype=False)
```

Checkpoint B: messy values (Step 3)

```
frame = pl.DataFrame({"ClientID": ["C1 ", "C2"], "CreatedAt": [pd.Timestamp("2026-08-03").to_pydatetime()] * 2,
                      "LedgerType": [" bill", "DEPOSIT"], "Debit": [5.0, 0.0], "Credit": [0.0, 9.0]})
fake = types.ModuleType("fake_loader")
fake.scan_ledger = lambda columns=None, only_new_appends=False: frame.lazy()
sys.modules["fake_loader"] = fake
with self.settings(LEDGER_MODULE="fake_loader"):
    rows, _ = get_ledger_slice(pd.Timestamp("2026-08-01"), pd.Timestamp("2026-09-01"),
                               clients=["C1"], types=["BILL"])
self.assertEqual(len(rows), 1)
```

Checkpoint C: notes (Step 4)

Same idea with a row whose CreatedAt is None: one row back, and `notes[0]` contains `"1 ledger rows have an unreadable CreatedAt"`. This one is already in `core/tests.py` as `test_unreadable_dates_are_reported`.

Checkpoint D: configurable loader (Step 5)

With `override_settings(LEDGER_MODULE="core.demo_ledger")`, `_module().__name__` equals `"core.demo_ledger"`; with `"fake_loader"` (after adding it to `sys.modules`), it equals that.

Checkpoint E: safe arguments (Step 6)

```
@override_settings(LEDGER_BASE_PATH="", LEDGER_APPEND_FOLDER="")
def test_only_accepted_arguments(self):
    def loader_a(columns=None, only_new_appends=False): ...
    def loader_b(base_path=None, append_folder=None, columns=None): ...
    self.assertEqual(set(_loader_kwargs(loader_a, columns=COLUMNS, only_new_appends=True)),
                     {"columns", "only_new_appends"})
    self.assertEqual(set(_loader_kwargs(loader_b, columns=COLUMNS, only_new_appends=True)), {"columns"})
    with self.settings(LEDGER_BASE_PATH=r"C:\data\Ledger.csv"):
        self.assertEqual(set(_loader_kwargs(loader_b, columns=COLUMNS)), {"columns", "base_path"})
```

Checkpoint F: the cache loads once (Step 7)

```
def test_cache_loads_once(self):
    calls = {"n": 0}
    def get_final_ledger(columns=None, as_pandas=False):
        calls["n"] += 1
        return demo_ledger.get_final_ledger(columns=columns, as_pandas=as_pandas)
    fake = types.ModuleType("fake_loader")
    fake.get_final_ledger = get_final_ledger           # no scan_ledger: the fallback path
    sys.modules["fake_loader"] = fake
    with self.settings(LEDGER_MODULE="fake_loader"):
        get_ledger_slice(*AUG, clients=demo_ledger.DEMO_CLIENTS[:2])
        get_ledger_slice(*AUG, clients=demo_ledger.DEMO_CLIENTS[2:4])
    self.assertEqual(calls["n"], 1)
```

For checkpoints B, E and F, put the tests in a `SimpleTestCase` class with a `tearDown` that calls `clear_cache()` and removes `"fake_loader"` from `sys.modules`.

Extension: one improvement to make yourself

Make `_loader_kwargs` pass every argument when the function accepts `**kwargs`. Hint: `any(p.kind is inspect.Parameter.VAR_KEYWORD for p in accepted.values())`. Write the test first: a loader defined as `def f(**kw)` should receive `columns`.

Explain it back

Answer each in one or two sentences, out loud or on paper, without looking. Answers are on the next page.

1. Why does the function return a tuple instead of just the rows?
2. What's the difference between a LazyFrame and a DataFrame, and where does reading actually happen?
3. Why is the condition built as keep = keep & ... rather than written in one line?
4. What would go wrong without .cast(pl.String) on ClientID?
5. Why pl.collect_all instead of two .collect() calls?
6. Why does _module() check folder not in sys.path?
7. What problem does inspect.signature solve here?
8. Why does an empty setting become None, and why does None matter?
9. Why does the cache need a lock, and why is the lock a module-level object?
10. Why does the fallback return led[keep].copy() and list(_cache['notes'])?

Answers

1. Warnings must reach the page. Returning (rows, notes) lets the view show them; printing would hide them in a server log.
2. A LazyFrame is a plan; a DataFrame holds data. Reading happens only at .collect() (or pl.collect_all).
3. The client and type filters are optional, so they're added only when given. The pattern grows the condition step by step.
4. IDs read as numbers would never equal the text IDs from the upload, so those clients would silently get no rows.
5. Both plans share one scan of the files. Two collects would read every file twice.
6. Otherwise every request would add the same folder again, and sys.path would grow without end.
7. Loaders accept different parameters. Asking the function which ones it has avoids TypeError for arguments it doesn't take.
8. Empty text means "not set". None is filtered out, so the loader's own default path is used instead of an empty path.
9. Two simultaneous requests would both load the ledger. The lock must be shared by every request, so it lives at module level.
10. So a caller changing its result can't change the cached ledger or the cached notes for the next request.

Appendix. The full file, annotated

Line numbers on the left; what each block is for in the table after the listing.

```
"""
Ledger rows for one report.

With a loader that has scan_ledger() (ledger_loader.py returns a Polars LazyFrame), only the rows a
report needs are read: the month, the clients and the trading types are filtered before anything is
collected, so the full ledger never sits in memory.

With an older loader that only has get_final_ledger(), the whole ledger is loaded once, kept for
LEDGER_CACHE_MINUTES, and filtered in pandas.
"""
import importlib
import inspect
import sys
import threading
import time
import warnings

import pandas as pd
import polars as pl
from django.conf import settings

COLUMNS = ("ClientID", "CreatedAt", "LedgerType", "Debit", "Credit")

_cache = {"ledger": None, "loaded_at": 0.0, "notes": []}
_lock = threading.Lock()


def _module():
    """Import the loader module named in settings, from LEDGER_MODULE_PATH if given."""
    folder = getattr(settings, "LEDGER_MODULE_PATH", "")
    if folder and folder not in sys.path:
        sys.path.insert(0, folder)
    return importlib.import_module(settings.LEDGER_MODULE)


def _loader_kwargs(func, **wanted):
    """Pass only the arguments this loader function accepts, and only paths that are set in settings."""
    wanted.setdefault("base_path", getattr(settings, "LEDGER_BASE_PATH", "") or None)
    wanted.setdefault("append_folder", getattr(settings, "LEDGER_APPEND_FOLDER", "") or None)
    accepted = inspect.signature(func).parameters
    return {k: v for k, v in wanted.items() if k in accepted and v is not None}


def get_ledger_slice(start, end, clients=None, types=None):
    """Rows with start <= CreatedAt < end, optionally only some clients and ledger types.

    Returns (rows as a pandas DataFrame, notes): notes are warnings worth showing on the page.
    """
    module = _module()
    if hasattr(module, "scan_ledger"):
        return _slice_lazy(module, start, end, clients, types)
    return _slice_from_cache(module, start, end, clients, types)


def _slice_lazy(module, start, end, clients, types):
    kwargs = _loader_kwargs(module.scan_ledger, columns=COLUMNS,
                            only_new_appends=getattr(settings, "LEDGER_ONLY_NEW_APPENDS", False))
    lf = module.scan_ledger(**kwargs)

    keep = ((pl.col("CreatedAt") >= pd.Timestamp(start).to_pydatetime())
            & (pl.col("CreatedAt") < pd.Timestamp(end).to_pydatetime()))
    if clients is not None:
```

```
        ids = sorted({str(c).strip() for c in clients if pd.notna(c)})
        keep = keep & pl.col("ClientID").cast(pl.String).str.strip_chars().is_in(ids)
    if types is not None:
        keep = keep & pl.col("LedgerType").cast(pl.String).str.strip_chars().str.to_uppercase().is_in(list(types))

    # One read of the files gives both the report's rows and the count of unreadable dates.
    rows, bad = pl.collect_all([lf.filter(keep),
                                lf.select(pl.col("CreatedAt").null_count().alias("n"))])
    notes = []
    n_bad = int(bad["n"][0])
    if n_bad:
        notes.append(f"{n_bad:,} ledger rows have an unreadable CreatedAt and were left out of every report.")
    return rows.to_pandas(), notes


def _slice_from_cache(module, start, end, clients, types):
    max_age = getattr(settings, "LEDGER_CACHE_MINUTES", 30) * 60
    with _lock:                                             # two requests at once load it only once
        if _cache["ledger"] is None or time.time() - _cache["loaded_at"] > max_age:
            kwargs = _loader_kwargs(module.get_final_ledger, columns=COLUMNS, as_pandas=True,
                                    only_new_appends=getattr(settings, "LEDGER_ONLY_NEW_APPENDS", False))
            with warnings.catch_warnings(record=True) as caught:
                warnings.simplefilter("always")
                led = module.get_final_ledger(**kwargs)
            if isinstance(led, pl.DataFrame):
                led = led.to_pandas()
            led["CreatedAt"] = pd.to_datetime(led["CreatedAt"])
            _cache.update(ledger=led, loaded_at=time.time(), notes=[str(w.message) for w in caught])
        led, notes = _cache["ledger"], list(_cache["notes"])

    keep = (led["CreatedAt"] >= start) & (led["CreatedAt"] < end)
    if clients is not None:
        keep &= led["ClientID"].astype(str).str.strip().isin({str(c).strip() for c in clients})
    if types is not None:
        keep &= led["LedgerType"].astype(str).str.strip().str.upper().isin(list(types))
    return led[keep].copy(), notes


def clear_cache():
    """Forget the cached full ledger (only used by loaders without scan_ledger)."""
    with _lock:
        _cache.update(ledger=None, loaded_at=0.0, notes=[])
```

Lines	Block	Purpose
1-9	`Module docstring`	What the file is for, and its two paths
11-20	`Imports`	Standard library first, then pandas/Polars, then Django
22	`COLUMNS`	The only columns any report needs; nothing else is read
24-25	`_cache, _lock`	Module-level state shared by every request
28-33	`_module()`	Which loader: settings + sys.path + importlib
36-41	`_loader_kwargs()`	Only accepted, non-empty arguments
44-52	`get_ledger_slice()`	The public contract; chooses the path
55-75	`_slice_lazy()`	Build the filter, collect once, report unreadable dates
78-98	`_slice_from_cache()`	Load once under a lock, cache, filter in pandas
101-104	`clear_cache()`	Reset for tests